

Real-World Drawbacks of First Normal Form (1NF)**Scenario: Employee Contact System**

After converting data into First Normal Form, an employee with multiple phone numbers is stored as follows:

EMP_ID	EMP_NAME	EMP_PHONE
2	Ravi	9876543210
2	Ravi	9123456789

1. Data Redundancy

The employee name *Ravi* is stored multiple times for the same employee.

Impact: In large organizations, repeated employee details such as name, department, and address waste storage space.

2. Update Anomaly

If Ravi changes his name to *Ravindra*, the update must be made in all rows.

Problem: If one row is not updated, the database will contain inconsistent employee records.

3. Deletion Anomaly

If Ravi's last phone number is deleted, all information about the employee is removed.

Problem: Employee data is lost even though the employee still exists.

4. Insertion Anomaly

Suppose a new employee *Suman* joins but has not yet been assigned a phone number.

Problem: The employee record cannot be inserted if *EMP_PHONE* is mandatory.

5. Performance Issues

An HR report requests: "*List all employees.*"

Problem: Duplicate employee rows are returned, requiring extra processing using *DISTINCT* or *GROUP BY*.

6. Poor Relationship Modeling

Phone numbers are a separate entity, but First Normal Form forces them into the same table.

Better Design:

- Employee table
- Phone table (linked using *EMP_ID*)

Conclusion: First Normal Form removes repeating groups but does not fully represent real-world relationships.

STEP 1: Table (Not in 1NF)

Suppose we have a Student Course table:

StudentID	StudentName	Courses
101	Rahul	DBMS, OS
102	Anjali	CN
103	Vikram	DBMS, CN, AI

Why this table is NOT in 1NF?

The **Courses** column contains multiple values in a single cell. For example, Rahul has "DBMS, OS" stored in one cell.

This is a **multi-valued attribute**.

First Normal Form (1NF) requires that each cell must contain only one atomic (indivisible) value.

STEP 2: Convert into First Normal Form (1NF)

To convert this table into 1NF, we must separate the multiple values into different rows.

Converted Table (1NF)

StudentID	StudentName	Course
101	Rahul	DBMS
101	Rahul	OS
102	Anjali	CN
103	Vikram	DBMS
103	Vikram	CN
103	Vikram	AI

Explanation

- Now each cell contains only one value.
- There are no repeating groups.
- Each row represents one student and one course.
- The table now follows First Normal Form (1NF).

Experiment No: 001**Aim**

To convert an unnormalized banking table into First Normal Form (1NF).

Problem Statement

Consider a Banking Management System that stores customer and account details in the following table:

Table: Customer_Accounts (Unnormalized Form)

Cst ID	Cst Name	Ph No	Acc No	Acc Typs
C101	Rahul Sharma	9876543210, 9123456780	A1001, A1002	Savings, Current
C102	Priya Singh	9988776655	A1003	Savings

The above table contains repeating groups and multiple values in single columns.

Tasks to Perform

- Identify the problems in the given table.
- Convert the table into First Normal Form (1NF).
- Create the normalized table using SQL.
- Insert appropriate records into the 1NF table.
- Display all records from the normalized table.

Problems Identified

- PhoneNumbers column contains multiple values.
- AccountNumbers column contains multiple values.
- AccountTypes column contains multiple values.
- Repeating groups violate 1NF.
- Values are not atomic.

Conversion to First Normal Form (1NF)

To convert into 1NF:

- Separate multiple values into individual rows.
- Ensure each column contains only one value.

Customer_Accounts (1NF)

Cst ID	Cst Name	ph no	Acc No	Acc Tpe
C101	Rahul Sharma	9876543210	A1001	Savings
C101	Rahul Sharma	9123456780	A1002	Current
C102	Priya Singh	9988776655	A1003	Savings

Now:

- Each column contains atomic values.
- No repeating groups.
- Each row represents a unique record.

SQL Implementation

Step 1: Create Table

```
CREATE TABLE Customer_Accounts_1NF (
    CustomerID VARCHAR(10),
    CustomerName VARCHAR(50),
    PhoneNumber VARCHAR(15),
    AccountNumber VARCHAR(10),
    AccountType VARCHAR(20),
    PRIMARY KEY (CustomerID, AccountNumber)
);
```

Step 2: Insert Records

```
INSERT INTO Customer_Accounts_1NF VALUES
('C101', 'Rahul Sharma', '9876543210', 'A1001', 'Savings'),
('C101', 'Rahul Sharma', '9123456780', 'A1002', 'Current'),
('C102', 'Priya Singh', '9988776655', 'A1003', 'Savings');
```

Step 3: Display Records

```
SELECT * FROM Customer_Accounts_1NF;
```

Expected Output

The resulting table satisfies First Normal Form (1NF):

- Each column contains atomic values.
- No repeating groups.
- Each row is unique.

Step 1: Unnormalized Table (UNF)

Table: Customer_Accounts (Before Normalization)

Cst ID	Cst Name	ph no	Acc no	Acc Typs
C101	Rahul Sharma	9876543210,	A1001,	Savings,
		9123456780	A1002	Current
C102	Priya Singh	9988776655	A1003	Savings

Problems in This Table:

- Multiple phone numbers in one column
- Multiple account numbers in one column
- Multiple account types in one column
- Not atomic values
- Violates 1NF rule

Step 2: Convert into First Normal Form (1NF)

Rule of 1NF:

- Each column must contain atomic (single) values
- No repeating groups
- Each row must be unique

Table: CustomerAccounts (After 1NF)

Cst ID	Cst Name	ph no	Acc no	Acc Typs
C101	Rahul Sharma	9876543210	A1001	Savings
C101	Rahul Sharma	9876543210	A1002	Current
C101	Rahul Sharma	9123456780	A1001	Savings
C101	Rahul Sharma	9123456780	A1002	Current
C102	Priya Singh	9988776655	A1003	Savings

What We Did:

- Removed multiple values from one column
- Created separate rows for each phone number and account
- Now every column has a single value
- Table follows 1NF

SQL Table Creation (1NF)

```
CREATE TABLE CustomerAccounts (
    CustomerID VARCHAR(10),
    CustomerName VARCHAR(50),
    PhoneNumber VARCHAR(15),
    AccountNumber VARCHAR(10),
    AccountType VARCHAR(20)
);
```